DBMON-05: Query Analysis

> **Series:** DBMON — Database Monitoring | **Notebook:** 5 of 7 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook provides deep query analysis techniques for identifying database performance problems. You will learn how to detect N+1 query anti-patterns, analyze query frequency and duration distributions, identify queries that may lack proper indexes, detect connection pool exhaustion, and use query normalization for effective grouping. These techniques apply across all database technologies.

Table of Contents

1. N+1 Query Detection
2. Query Frequency Analysis
3. Duration Distribution Analysis
4. Missing Index Detection
5. Connection Pool Analysis
6. Query Normalization and Grouping
7. Optimization Prioritization
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there)
OneAgent	Deployed on application hosts with database clients
Permissions	`storage:spans:read`
Data	Significant application traffic with diverse database query patterns
Prior Reading	DBMON-01 through DBMON-03 for database monitoring context

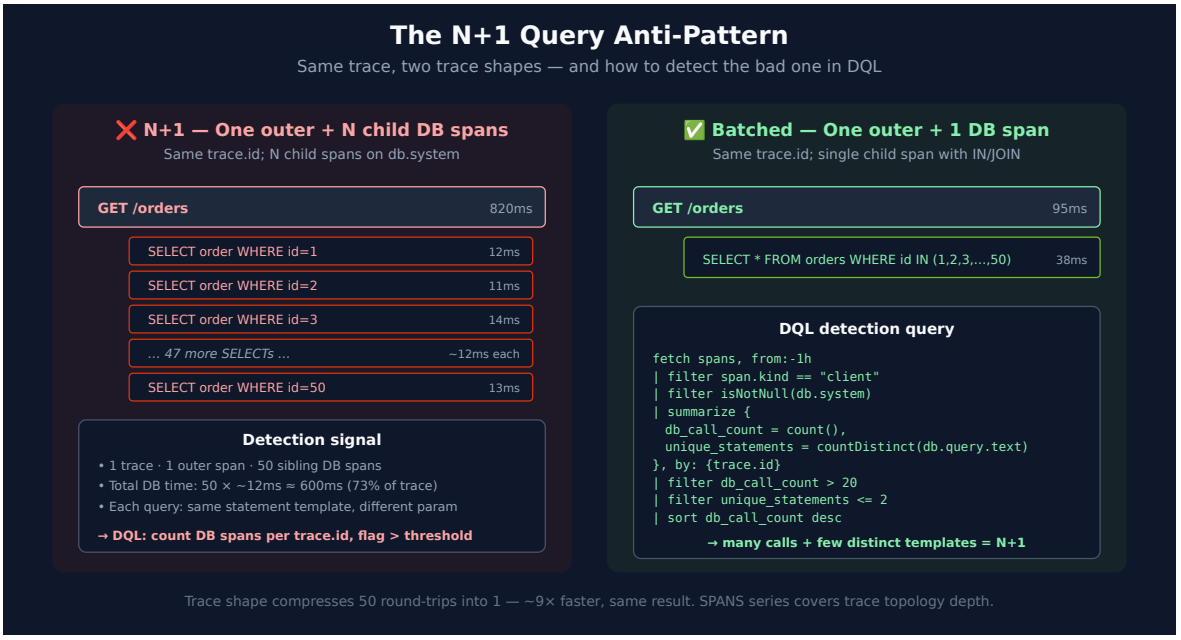
1. N+1 Query Detection

The N+1 query problem occurs when an application executes one query to fetch a list of items, then executes N additional queries to fetch related data for each item. This is the most common database anti-pattern and can cause severe performance degradation.

Detection Strategy

N+1 patterns have these characteristics:
- A single normalized query pattern is executed many times within a short window
- The queries share the same parent span (same HTTP request or transaction)
- Individual query execution times are fast, but total time is high due to volume

We detect this by looking for query patterns with unusually high call counts per trace.



In community practice the thresholds in this notebook are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Detect N+1 patterns — query patterns with high repetition per trace
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    calls_per_trace = count(),
    total_ms = sum(duration) / 1ms,
    avg_ms = avg(duration) / 1ms
}, by:{trace.id, db.query.text, db.system}
| filter calls_per_trace &gt;= 10
| sort calls_per_trace desc
| limit 25
```

The query above shows query patterns that are repeated 10 or more times within a single trace. A high `calls_per_trace` value (especially 50+) strongly suggests an N+1 pattern.

> **Tip:** Focus on patterns where `avg_ms` is low but `total_ms` is high. This indicates many fast individual queries that accumulate to a significant total. The fix is typically to replace N individual queries with a single batch query using `IN` clauses or joins.

```
// Aggregate N+1 candidates — which query patterns are most frequently
repeated?
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize calls_per_trace = count(), by:{trace.id, db.query.text, db.system}
| filter calls_per_trace &gt;= 10
| summarize {
    affected_traces = count(),
    avg_calls_per_trace = avg(calls_per_trace),
    max_calls_per_trace = max(calls_per_trace)
}, by:{db.query.text, db.system}
| sort affected_traces desc
| limit 15
```

2. Query Frequency Analysis

Understanding which queries run most often helps prioritize optimization. A query that runs millions of times per hour has far more impact than a slow query that runs once a day.

```
// Top 20 most frequent queries — highest call volume
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    call_count = count(),
    total_time_ms = sum(duration) / 1ms,
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by:{db.query.text, db.system}
| sort call_count desc
| limit 20
```

```
// Query frequency trend — detect changing patterns over 6 hours
fetch spans, from:-6h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| makeTimeseries {
    total_queries = count(),
    unique_patterns = countDistinct(db.query.text)
  }, interval:15m
```

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Queries called by the most services — shared queries are high-impact
optimization targets
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    call_count = count(),
    calling_services = countDistinct(dt.service.name),
    avg_ms = avg(duration) / 1ms
}, by:{db.query.text, db.system}
| filter calling_services &gt;= 2
| sort calling_services desc
| limit 15
```

3. Duration Distribution Analysis

Analyzing how query durations are distributed reveals bimodal patterns (e.g., cached vs uncached responses), tail latency problems, and outlier behavior.

```
// Duration percentile breakdown by database system
fetch spans, from:-1h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    p50_ms = percentile(duration, 50) / 1ms,
    p90_ms = percentile(duration, 90) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    p99_ms = percentile(duration, 99) / 1ms,
    max_ms = max(duration) / 1ms,
    total = count()
}, by:{db.system}
| fieldsAdd tail_ratio = round(p99_ms / p50_ms, decimals:1)
| sort total desc
```

The `tail_ratio` column shows how much worse the 99th percentile is compared to the median. A ratio above 10 suggests significant tail latency — a small percentage of queries take disproportionately long.

```
// Latency bucket distribution — visualize where queries cluster
// The numeric prefix makes the buckets sort in latency order rather than
alphabetically.
fetch spans, from:-1h
| filter isNotNull(db.system)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| fieldsAdd duration_ms = duration / 1ms
| fieldsAdd bucket = if(duration_ms &lt; 0.1, then:"1: &lt;0.1ms",
    else:if(duration_ms &lt; 1, then:"2: 0.1-1ms",
    else:if(duration_ms &lt; 10, then:"3: 1-10ms",
    else:if(duration_ms &lt; 100, then:"4: 10-100ms",
    else:if(duration_ms &lt; 1000, then:"5: 100ms-1s",
    else:"6: &gt;1s")))))
| summarize query_count = count(), by:{db.system, bucket}
| sort db.system asc, bucket asc
```

4. Missing Index Detection

Queries that consistently have high latency and high call count may indicate missing database indexes. Spans do not show index usage, so the queries below identify candidates by correlating query patterns with performance characteristics.

For plan-level evidence, use the **Databases** app on instances monitored by a database extension: open the statement and request its execution plan, which shows the indexes, joins and scans the engine actually used. Treat the heuristics below as a way to choose which statements to look at.

> **Sources:** Database statements and execution plans (DT docs) — *"Database extensions collect the statements per database instance, ranked by resource consumption and execution time."* and *"It shows which indexes, joins, scans, and other operations the engine uses, and the order in which it performs them."*

Heuristics for Missing Indexes

Signal	What It Suggests
SELECT with high avg latency and high volume	Table scan on frequently-read table
P95 >> P50 for the same query pattern	Some executions hit different data sizes
Query latency increasing over time	Table growing without proper indexing
High variance in execution time	Inconsistent query plans

```
// Index candidate detection — high-volume queries with high latency and
variance
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter isNotNull(db.operation.name) and db.operation.name == "SELECT"
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p50_ms = percentile(duration, 50) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    stddev_ms = stddev(duration / 1ms)
}, by:{db.query.text, db.system}
| filter call_count &gt;= 50 and avg_ms &gt; 10
| fieldsAdd variance_ratio = round(stddev_ms / avg_ms, decimals:2)
| sort avg_ms desc
| limit 15
```

```
// Detect queries getting slower over time — potential index degradation
fetch spans, from:-24h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter isNotNull(db.operation.name) and db.operation.name == "SELECT"
| makeTimeseries {
    avg_ms = avg(duration / 1ms),
    call_count = count()
  }, by:{db.system}, interval:1h
```

5. Connection Pool Analysis

Connection pool exhaustion causes application threads to wait for available connections, leading to increased response times and timeouts. Spans do not carry pool counters, so the queries below look for its symptoms: call volume per service, connection errors, and error-rate spikes. Pool metrics themselves come from runtime metrics (for example JMX or .NET connection-pool counters), not from spans.

> **Measure the pool directly on Java (updated 11/11/2026).** For Java application servers and pool libraries, the Dynatrace Hub has **Connection Pools** extensions built on Extensions 2.0, with pages for JBoss, WebLogic, WebSphere, WebSphere Liberty, Apache Commons and C3P0. Each one reads pool counters over JMX through OneAgent. You activate the OneAgent feature *Java Metric Extensions 2.0 (JMX)* and add the extension, and it ships an overview dashboard. The C3P0 extension, for example, reports `c3p0.connectionPool.maxPoolSize`, `c3p0.connectionPool.numConnections` and `c3p0.connectionPool.numBusyConnections`. With busy connections close to the maximum, the symptom queries below become a confirmed diagnosis. The extensions are free to use, but the metrics they ingest are billed (Metrics powered by Grail on DPS, DDUs on a classic license). Each page gives a per-pool data-point formula, so size the cost before you enable every feature set.
>
> **Sources:** Connection Pools C3P0 extension (DT docs) — *"Several built-in OneAgent extensions provide an insight into the connection pool performance and issues such as connection leaks."*, *"Activate the OneAgent feature Java Metric Extensions 2.0 (JMX)."*; Connection Pools WebLogic extension (DT docs) — *"There is no charge to use the extension. You are only charged for the data that the extension ingests."*; Connection Pools JBoss extension (DT docs) — *"This extension is built on top of the new Extension 2.0 Framework."*; re-read 11/11/2026. **Derived:** reading busy against

> maximum connections to confirm exhaustion combines the C3P0 metric list with this section's symptom queries.

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Database call volume per service — candidates for connection pressure
// This counts calls in the window; it is not concurrency (calls in flight at
the same moment).
fetch spans, from:-1h
| filter isNotNull(db.system)
| summarize {
    call_count = count(),
    unique_db_targets = countDistinct(server.address),
    avg_ms = avg(duration) / 1ms
}, by:{dt.service.name, db.system}
| sort call_count desc
| limit 20
```

```
// Database connection errors — timeout and connection refused patterns
// span.status_message is experimental and is often empty on failed DB spans;
the error
// text is recorded as an exception event in span.events. expand counts
exception events,
// so a span with two exception events counts twice.
fetch spans, from:-6h
| filter isNotNull(db.system) and span.status_code == "error"
| expand span.events
| filter isNotNull(span.events[exception.type])
| fieldsAdd exception_type = span.events[exception.type], exception_message =
span.events[exception.message]
| summarize error_count = count(), by:{db.system, server.address,
exception_type, exception_message}
| sort error_count desc
| limit 20
```

```
// Database error rate trend — detect connection pool exhaustion episodes
fetch spans, from:-6h
| filter isNotNull(db.system)
| makeTimeseries {
    total = count(),
    errors = countIf(span.status_code == "error", default:0)
  }, interval:5m
```

6. Query Normalization and Grouping

`db.query.text` groups well only when identical query shapes produce identical text. Two things decide that:

- **Parameterized statements** (prepared statements, ORMs) already carry placeholders — `?`, `@p0`, `$1` — so every execution of the same statement has the same text.
- **`WHERE`-clause literals are masked** by Dynatrace: *"Literals that are part of the WHERE clause of an SQL statement are replaced with ******"*.**

SQL built by string concatenation with inline values outside the `WHERE` clause (for example `INSERT … VALUES (42, 'x')`) produces a different text per execution — that is what the one-off query below surfaces.

Statement as executed	What `db.query.text` looks like
`SELECT * FROM users WHERE id = ?` (prepared)	unchanged — one pattern
`SELECT * FROM users WHERE id = 42` (inline literal)	the `WHERE` literal masked — one pattern
`INSERT INTO orders (id, amount) VALUES (1, 99.50)` (inline literals)	literals outside `WHERE` remain — one pattern per value

> **Sources:** Personal data captured by Dynatrace (DT docs) — *"Literals that are part of the WHERE clause of an SQL statement are replaced with ***** , for example, WHERE userId = '*******' ."*

```
// Query pattern diversity — how many unique normalized patterns per database?
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    unique_patterns = countDistinct(db.query.text),
    total_calls = count()
}, by:{db.system, db.namespace}
| fieldsAdd avg_calls_per_pattern = round(toDouble(total_calls) /
toDouble(unique_patterns), decimals:1)
| sort total_calls desc
```

```
// Identify one-off queries — patterns called only once (may indicate dynamic
SQL)
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize call_count = count(), by:{db.query.text, db.system}
| filter call_count == 1
| summarize one_off_count = count(), by:{db.system}
| sort one_off_count desc
```

> **Warning:** A high number of one-off query patterns suggests dynamic SQL generation or poor query parameterization. This prevents the database from caching execution plans and can degrade performance significantly.

7. Optimization Prioritization

Not all slow queries are worth optimizing. Use the **impact score** to prioritize: combine call frequency, average duration, and error rate into a single ranking.

```
// Query optimization priority — ranked by total time impact
fetch spans, from:-1h
| filter isNotNull(db.system) and isNotNull(db.query.text)
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))  // statements only (DBMON-01 § 4)
| summarize {
    call_count = count(),
    total_time_ms = sum(duration) / 1ms,
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    error_count = countIf(span.status_code == "error")
}, by:{db.query.text, db.system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
toDouble(call_count)) * 100, decimals:2)
| sort total_time_ms desc
| limit 20
```

The `total_time_ms` column represents the cumulative time spent on each query pattern. Optimizing the top entries yields the greatest overall performance improvement.

In community practice, the total-time ranking is read with these heuristics — they suggest a cause, they do not establish one:

Priority	Criteria	Action
Critical	High total time + high error rate	Fix errors first, then optimize
High	High total time + high call count + low avg	Likely N+1 — batch the queries
Medium	High total time + low call count + high avg	Likely missing index — add index
Low	Low total time regardless of avg	Minimal impact — defer optimization

8. Summary and Next Steps

In this notebook you learned:

- How to detect N+1 query anti-patterns using per-trace call count analysis
- Query frequency analysis to identify the most common database operations
- Duration distribution analysis to reveal tail latency and bimodal patterns
- Heuristic-based missing index detection using latency and variance metrics
- Connection pool pressure analysis through call volume and error monitoring
- Query normalization concepts and one-off query detection
- Impact-based optimization prioritization

Next Steps

- **DBMON-06: Dashboards and Alerting** — Build database monitoring dashboards and configure alerting rules

Where to Go Deeper

- **SPANS series** (9 notebooks) — Span fundamentals, trace topology, parent/child span relationships (foundational for N+1 detection)
- **AIOPS series** (8 notebooks) — Davis anomaly detection on connection-pool exhaustion, slow-query rate, query-pattern shifts
- **WFLOW series** (12 notebooks) — Workflow-driven alert routing for DB-engineering teams

- **DASH series** (8 notebooks) — Impact-ranking dashboard tiles (rank queries by total impact, not average)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.